

Curso online: **Programación orientada a objetos: Java avanzado**

Pruebas Unitarias

Autores

Jorge Molinero Muñoz

Miguel Sallent Sánchez

ÍNDICE

1. Introducción	3
2. Creación de una prueba unitaria con JUnit en Eclipse.....	3
3. Assertions.....	13
4. Métodos de ciclo de vida	14
5. Las pruebas unitarias en las metodologías de desarrollo de software	15
6. Ejercicio Resuelto	16

1. Introducción

En el desarrollo de software uno de los pilares fundamentales es la realización de pruebas. En este tema veremos algunas utilidades para facilitar esta tarea.

Las pruebas unitarias consisten en pruebas que se realizan sobre las unidades más pequeñas de la aplicación, como puede ser un método o función en un lenguaje orientado a procedimientos, o una clase en un lenguaje orientado a objetos.

Los lenguajes orientados a objetos como Java encajan particularmente bien con las pruebas unitarias, ya que habitualmente separan completamente la definición de los métodos por un lado y su implementación por otro (encapsulación). Así pues, tendremos una serie de pruebas unitarias que probarán todos los casos posibles de un método, y si cambiamos la implementación de dicho método, bastará con ejecutar de nuevo todas las pruebas para verificar que la nueva implementación funciona correctamente.

Por ejemplo, imaginemos que tenemos una aplicación que se conecta a una base de datos, formada por una serie de clases que se encargan de conectarse a la base de datos y realizar las consultas. Diseñamos una serie de pruebas unitarias, lo ideal sería una prueba unitaria por cada consulta a la base de datos que se realice. Durante el mantenimiento evolutivo de la aplicación se advierte que es necesario modificar el modelo de datos, por ejemplo, dividiendo una tabla en dos, añadiendo o quitando columnas, modificando alguna vista, etc. Una vez hechos los cambios, ejecutaremos todas las pruebas unitarias y de esta forma nos aseguramos de que todas las consultas siguen funcionando correctamente, sin necesidad de volverlas a probar una a una como usuario desde la aplicación.

Existen diversos frameworks para la realización de pruebas unitarias, en este tema veremos el framework JUnit.

2. Creación de una prueba unitaria con JUnit en Eclipse

A continuación, mostraremos cómo utilizar JUnit en un proyecto sencillo en Eclipse, mediante un ejemplo.

En primer lugar crearemos un proyecto nuevo. Crearemos un paquete “tema7” y en él crearemos una clase CuentaVocales, con un método estático que simplemente contará el número de vocales de una cadena de texto. Aprovecharemos lo que ya conocemos de streams y expresiones lambda:

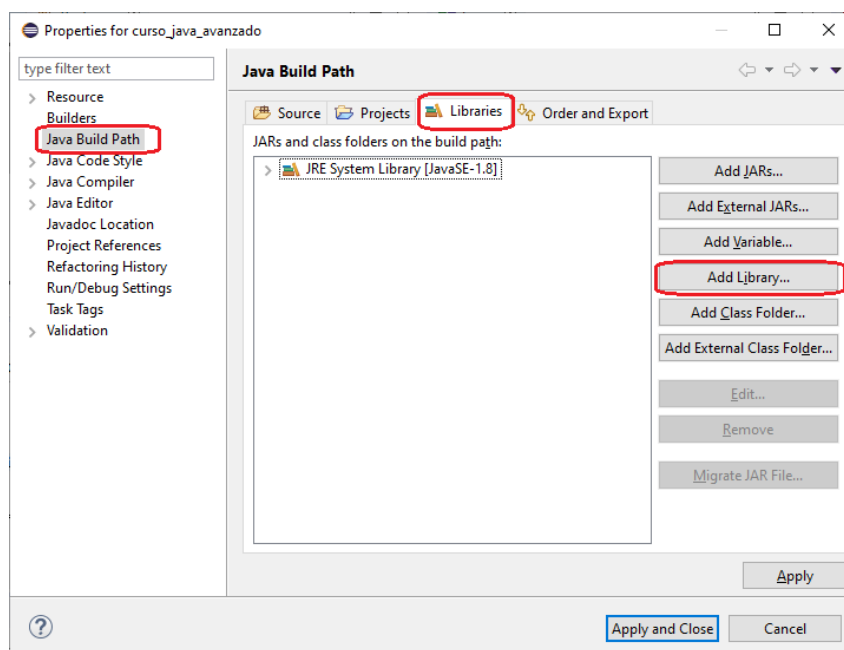
```
package tema7;

public class CuentaVocales {

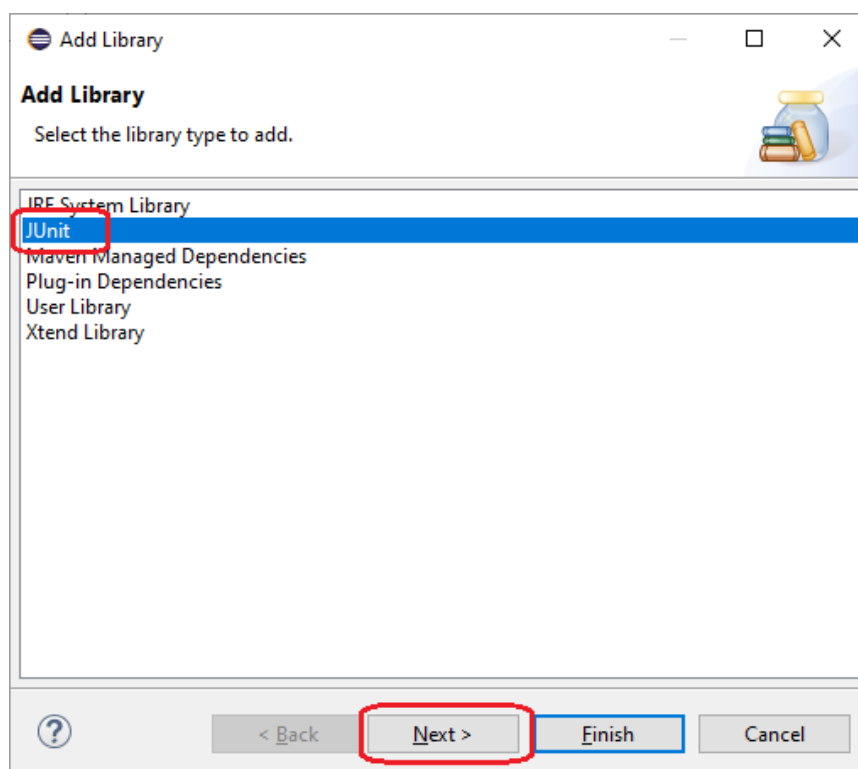
    public static long cuentaVocales(String s)
    {
        return s.chars().filter(
            c -> c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u'
            || c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U'
        ).count();
    }
}
```

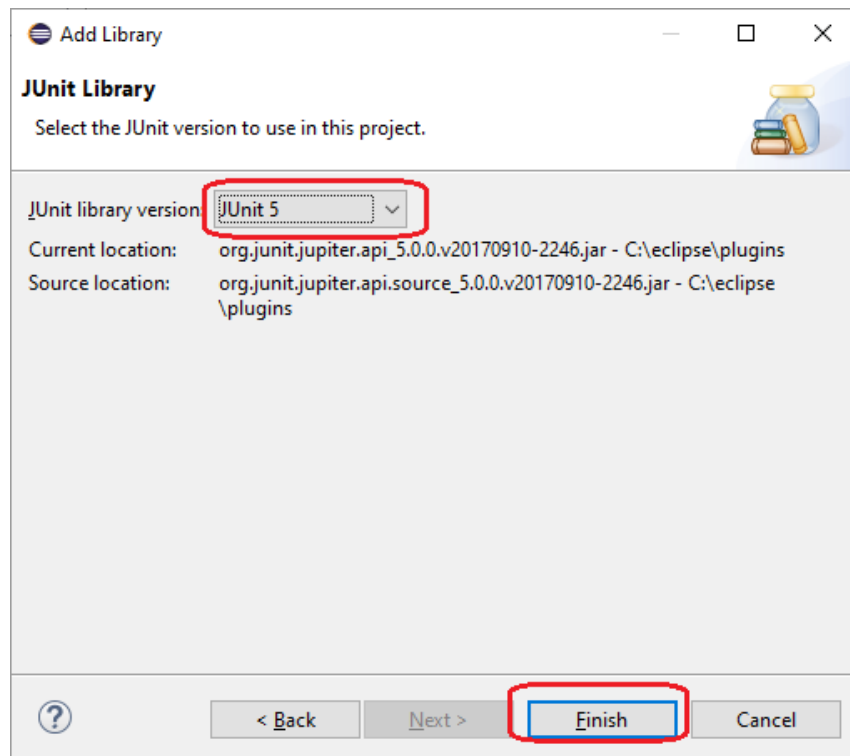
A continuación, si no conociéramos nada de pruebas unitarias, crearíamos una clase con un método main para probar nuestro método. En lugar de ello diseñaremos e implementaremos las pruebas unitarias.

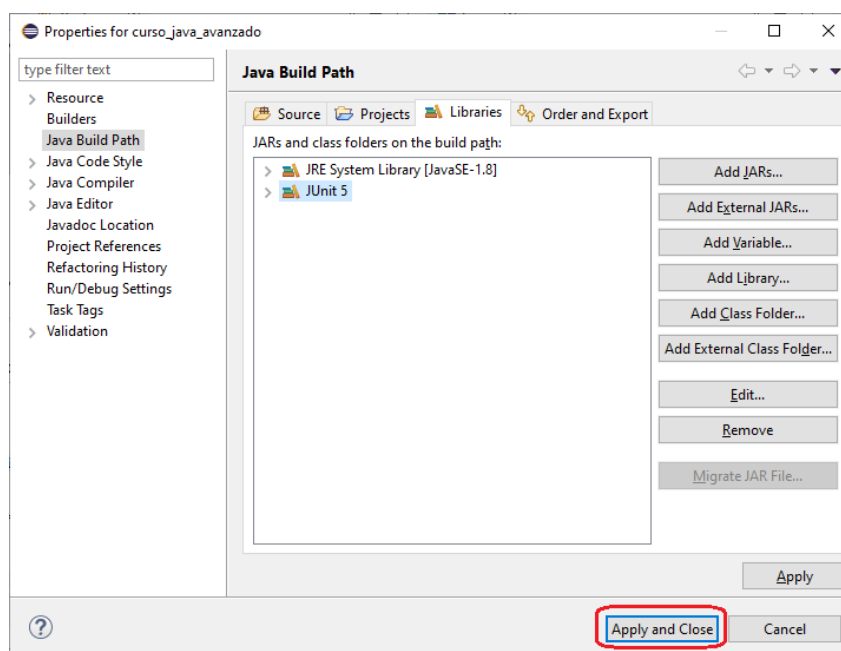
En primer lugar, añadiremos la librería junit al proyecto, para ello, en el menú seleccionaremos Project -> Properties y a continuación Java Build Path. Pulsaremos en la pestaña Libraries y en el botón “Add Library...”:



Seleccionaremos la librería junit, versión 5 (también valdría la versión 4, aunque el código que genera Eclipse varía ligeramente):







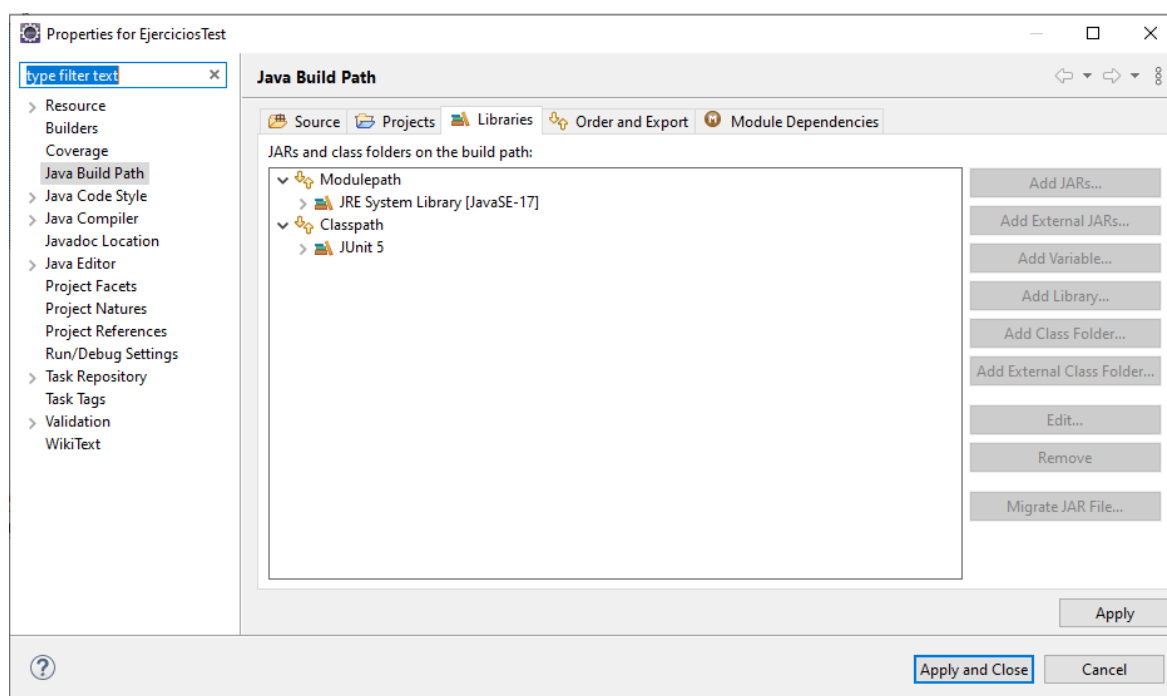
¡¡Ojo!! A partir de Java 9, Eclipse contempla el concepto de módulo.

Un módulo es un elemento o artefacto que agrupa código y recursos, de forma semejante a las librerías tradicionales, pero incluyendo un descriptor que restringe el acceso a sus paquetes, y describe sus dependencias.

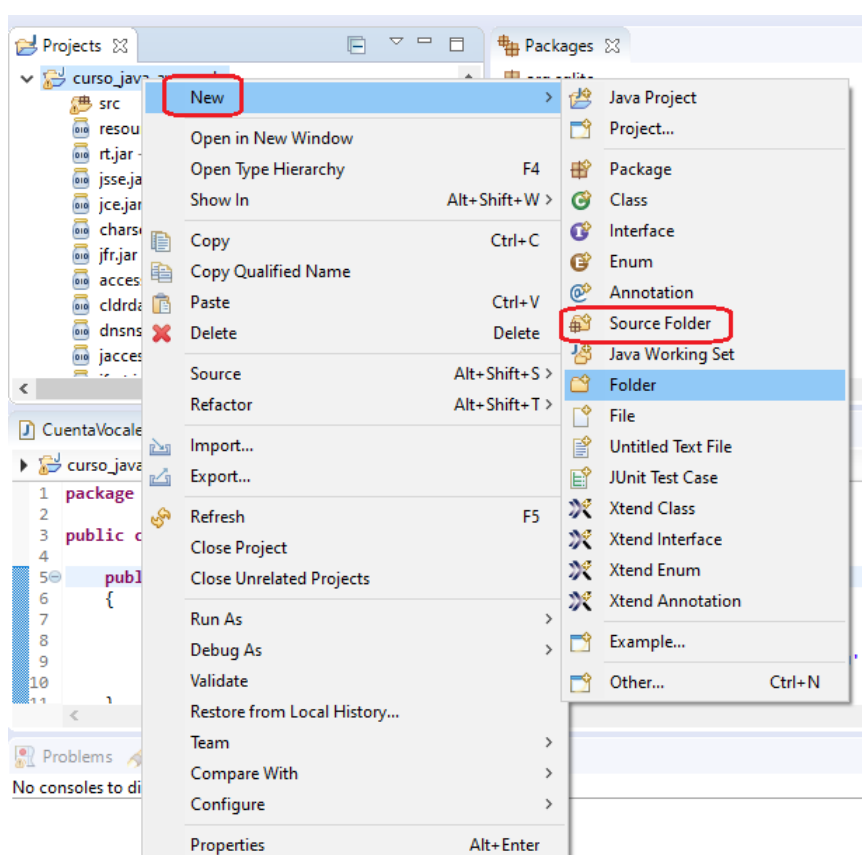
El descriptor de un módulo se escriben en un fichero module-info.java y contiene:

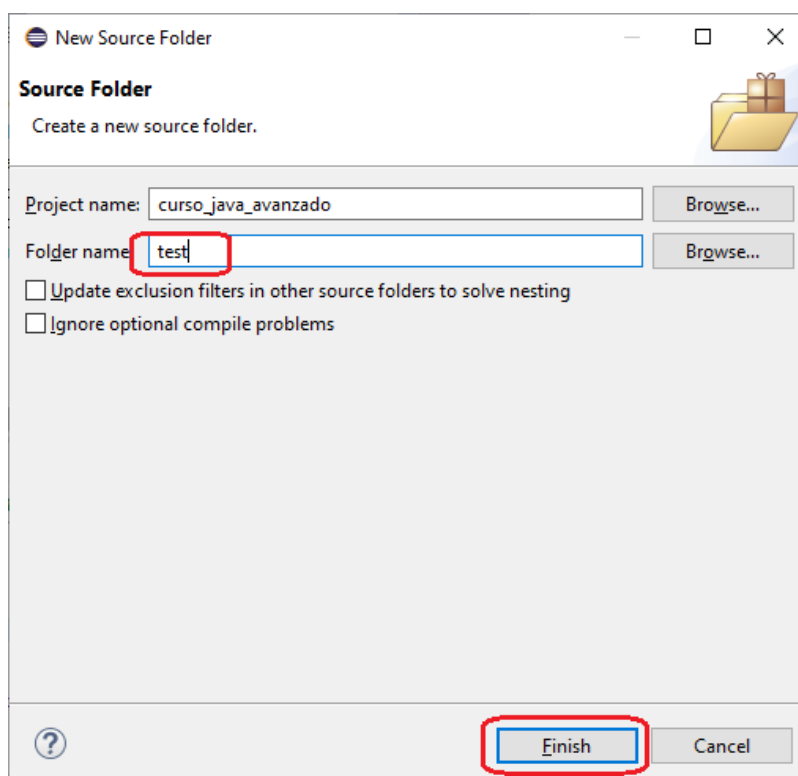
- Nombre del módulo
- Paquetes expuestos
- Dependencias con otros módulos
- Servicios consumidos e implementados

En el caso de estar utilizando un proyecto en Eclipse con una versión superior a Java 9, hay que tener cuidado de incorporar la librería al classpath, no el ModulePath, ya que si se incorpora al ModulePath únicamente tendrán visibilidad a dicha librería desde el módulo que estemos creando.

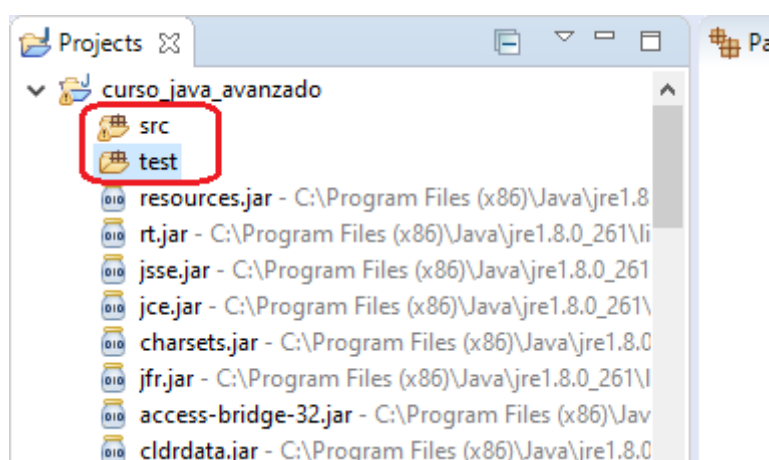


Para las pruebas unitarias tendremos que crear una serie de clases. Por regla general el código de las pruebas unitarias se mantiene separado del resto del código, ya que este código no forma parte de la aplicación y no se incluye cuando se compila la aplicación para su distribución o empaquetado. Para ello crearemos una carpeta “test”, pulsando con el botón derecho del ratón sobre el proyecto y utilizando la opción New -> Source Folder:



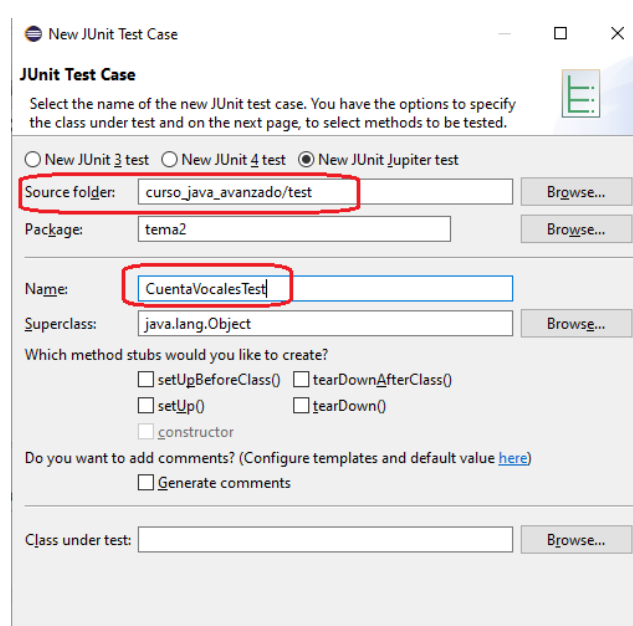
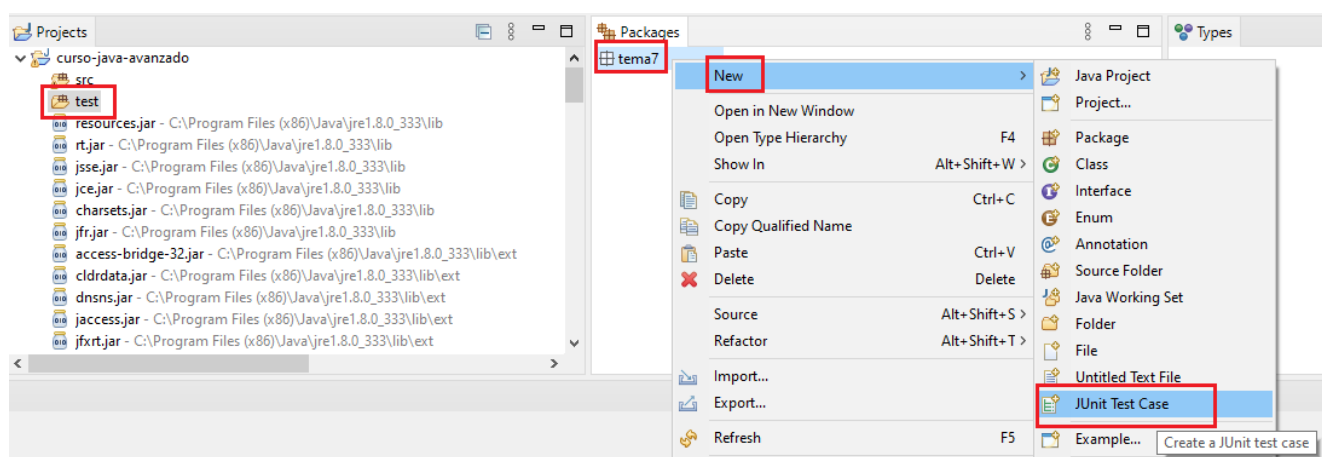


Veremos que ahora tenemos dos carpetas: utilizaremos la carpeta src para el código de la aplicación, y la carpeta test para las pruebas unitarias:



También se podría poner todo el código de las pruebas en un proyecto separado, depende de la estructura de la aplicación y de cómo se vaya a realizar el empaquetado o despliegue. Pero lo que sí debe hacerse siempre es poner el código de pruebas separado del de la aplicación.

A continuación, crearemos un paquete con el mismo nombre que hemos dado para la clase, "tema7", pero dentro de la carpeta test, y una clase que llamaremos CuentaVocalesTest. Para ello, en vez de utilizar la opción New -> Class, utilizaremos la opción New -> Junit Test Case:



En realidad un Test Case no es más que una clase Java, con ciertas particularidades.

No es necesario que la clase con las pruebas unitarias esté en el mismo paquete que la clase probada, pero ayuda a mantener las pruebas ordenadas. Además permite a la prueba unitaria acceder a métodos *protected*, ya que estos métodos son accesibles desde clases del mismo paquete, lo que nos permite también probar estos métodos.

A continuación abrimos la clase que hemos creado y editamos el método que realizará la prueba, de la siguiente manera:

```
package tema7;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class CuentaVocalesTest {

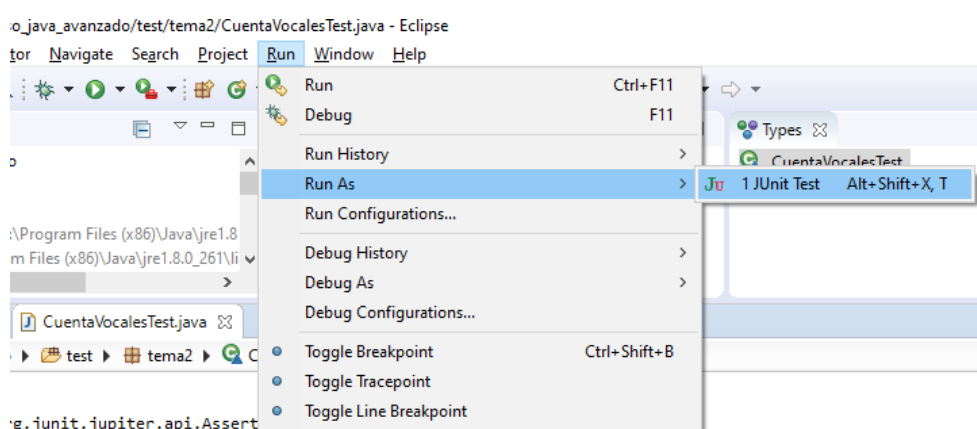
    @Test
```

```
void test() {
    String s = "Cadena de prueba";
    long cuenta = CuentaVocales.cuentaVocales(s);
    assertEquals(7, cuenta);
}
```

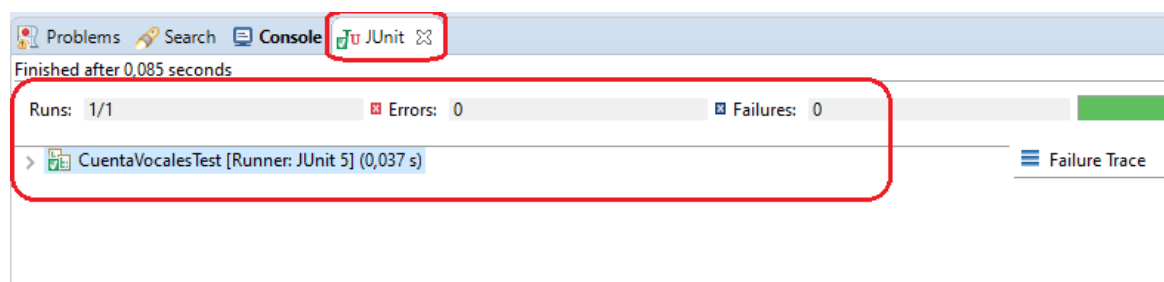
```
}
```

Los métodos “assert” son los que nos permiten indicar cuál es el resultado esperado. Por ejemplo, el método assertEquals toma dos argumentos: el primero, “expected”, indica el valor esperado. El segundo, “actual”, indica el valor calculado. Por tanto la sentencia “assertEquals(7, cuenta)” se lee “comprueba que la variable *cuenta* toma el valor 7”.

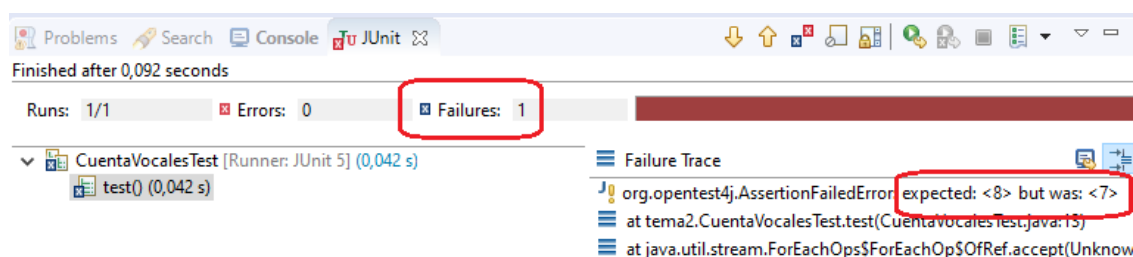
A continuación ejecutamos la prueba, para ello en el menú seleccionaremos la opción Run -> Run As -> JUnit Test:



En la parte inferior veremos el resultado, en este caso la prueba ha sido correcta:



Si la prueba hubiera sido incorrecta nos habría mostrado algo como esto:



Nos indica por tanto que “el valor esperado era 8 pero el obtenido fue 7”.

En una situación real habría que pensar en todos los casos posibles, especialmente los que podrían ser problemáticos, e implementar un método de prueba para cada uno. En este caso podríamos pensar por ejemplo en las siguientes pruebas, siempre tratando de cubrir todos los casos posibles:

- Una cadena cualquiera, como la del ejemplo anterior
- Una cadena vacía
- Una cadena null
- Una cadena con las cinco vocales, tanto mayúsculas como minúsculas
- Una cadena con las cinco vocales con acentos, tanto mayúsculas como minúsculas
- Una cadena con las cinco vocales con diéresis, tanto mayúsculas como minúsculas

Obsérvese que, para que las pruebas unitarias sean realmente útiles, deben ser exhaustivas. Además nos ayudan a hacer una reflexión sobre los casos problemáticos que a lo mejor no habíamos considerado al implementar el método.

El código de las pruebas sería algo así:

```
package tema7;

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;

class CuentaVocalesTest {

    @Test
    void test() {
        String s = "Cadena de prueba";
        long cuenta = CuentaVocales.cuentaVocales(s);
        assertEquals(7, cuenta);
    }

    @Test
    void testVacía() {
        String s = "";
        long cuenta = CuentaVocales.cuentaVocales(s);
        assertEquals(0, cuenta);
    }

    @Test
    void testNull() {
        String s = null;
        long cuenta = CuentaVocales.cuentaVocales(s);
        assertEquals(0, cuenta);
    }

    @Test
    void testTodasSinAcentos() {
        String s = "tatetitotuTATETITOTU";
        long cuenta = CuentaVocales.cuentaVocales(s);
        assertEquals(10, cuenta);
    }

    @Test
    void testTodasConAcentos() {
        String s = "áéíóúÁÉÍÓÚ";
        long cuenta = CuentaVocales.cuentaVocales(s);
    }
}
```

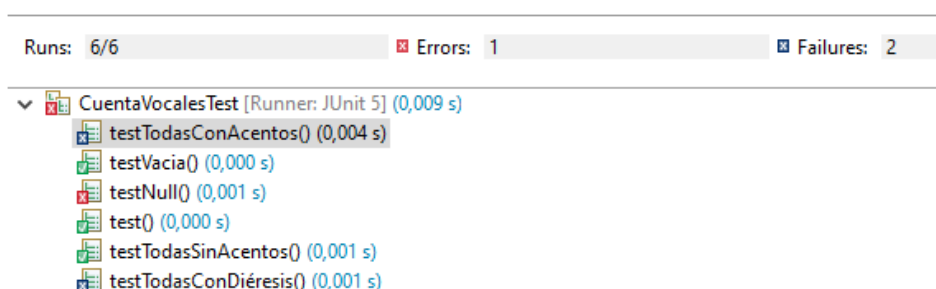
```

        assertEquals(10, cuenta);
    }

    @Test
    void testTodasConDiéresis() {
        String s = "äëïöüÄËÏÖÜ";
        long cuenta = CuentaVocales.cuentaVocales(s);
        assertEquals(10, cuenta);
    }
}

```

Cada uno de los métodos de prueba debe tener la anotación @Test. Ejecutamos las pruebas y observamos lo siguiente:



Observamos que de 6 pruebas, tres se han superado con éxito, una indica Error, y dos indican Failure.

“Error” representa un error no esperado, como por ejemplo una excepción NullPointerException (es el caso de testNull).

Sin embargo “Failure” indica un error que ha sido generado por una sentencia assert, donde el resultado no ha sido el esperado, a pesar de haberse ejecutado sin errores. En este caso, los tests testTodasConAcentos y testTodasConDiéresis no provocan error, pero el resultado del método es 0 en vez del correcto.

Parece que la primera implementación que hicimos del método cuentaVocales no era muy robusta, ya que no tuvimos en cuenta algunos casos posibles. Vemos que las pruebas unitarias nos ayudan a crear código robusto.

A continuación modificamos el método en la clase CuentaVocales para corregir todos los fallos:

```

package tema7;

public class CuentaVocales {

    public static long cuentaVocales(String s)
    {
        long result = 0;
        if (s != null) {
            return s.chars().filter(
                c -> c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u'
                || c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U'
                || c == 'á' || c == 'é' || c == 'í' || c == 'ó' || c == 'ú'
                || c == 'Á' || c == 'É' || c == 'Í' || c == 'Ó' || c == 'Ú'
                || c == 'ä' || c == 'ë' || c == 'ï' || c == 'ö' || c == 'ü'
                || c == 'Ä' || c == 'Ë' || c == 'I' || c == 'Ö' || c == 'Ü'
            ).count();
        }
        return result;
    }
}

```

```

    }
}

```

Si ahora ejecutamos las pruebas unitarias veremos que todas se ejecutan correctamente.

También observamos que, para cada test, Eclipse nos indica el tiempo que ha tardado en ejecutarse. Esto nos puede servir para detectar los cuellos de botella de la aplicación en relación al rendimiento, así como para optimizar tiempos de tareas pesadas (por ejemplo, podría servir para optimizar una consulta SQL a la base de datos).

3. Assertions

En el capítulo anterior hemos utilizado el método `assertEquals`. Este método está sobrecargado, de forma que puede tomar diversos argumentos: `int`, `long`, `Object`, etc. Nos permite por tanto comparar variables de distintos tipos.

Algunos otros métodos `assert` son:

- `assertNotEquals`: comprueba que dos elementos no sean iguales
- `assertTrue`: comprueba que el argumento sea verdadero
- `assertFalse`: comprueba que el argumento sea falso
- `assertNull`: comprueba que el argumento sea null
- `assertNotNull`: comprueba que el argumento no sea null

Para realizar un plan de pruebas completo, no hay que limitarse con probar las situaciones controladas, también se deben probar los posibles casos de error. Por ejemplo, supongamos que definimos nuestro método `cuentaVocales` de forma que debe lanzar la excepción `NullPointerException` si el argumento es null. Este caso se podría probar de la siguiente manera:

```

@Test
void testNullException() {
    String s = null;
    try
    {
        long cuenta = CuentaVocales.cuentaVocales(s);
        fail("Se debería haber lanzado una excepción NullPointerException");
    }
    catch (NullPointerException e)
    {
    }

    catch (Exception e)
    {
        fail("Se ha lanzado una excepción pero no es de tipo NullPointerException");
    }
}

```

Utilizamos el método *fail* para indicar una situación no esperada. En el ejemplo anterior, si no se lanza una excepción se invocará el método *fail* y el resultado de la prueba será `Failure`. Lo mismo ocurre si se lanza una excepción pero no es de tipo `NullPointerException`.

También se podría haber implementado de la siguiente manera equivalente:

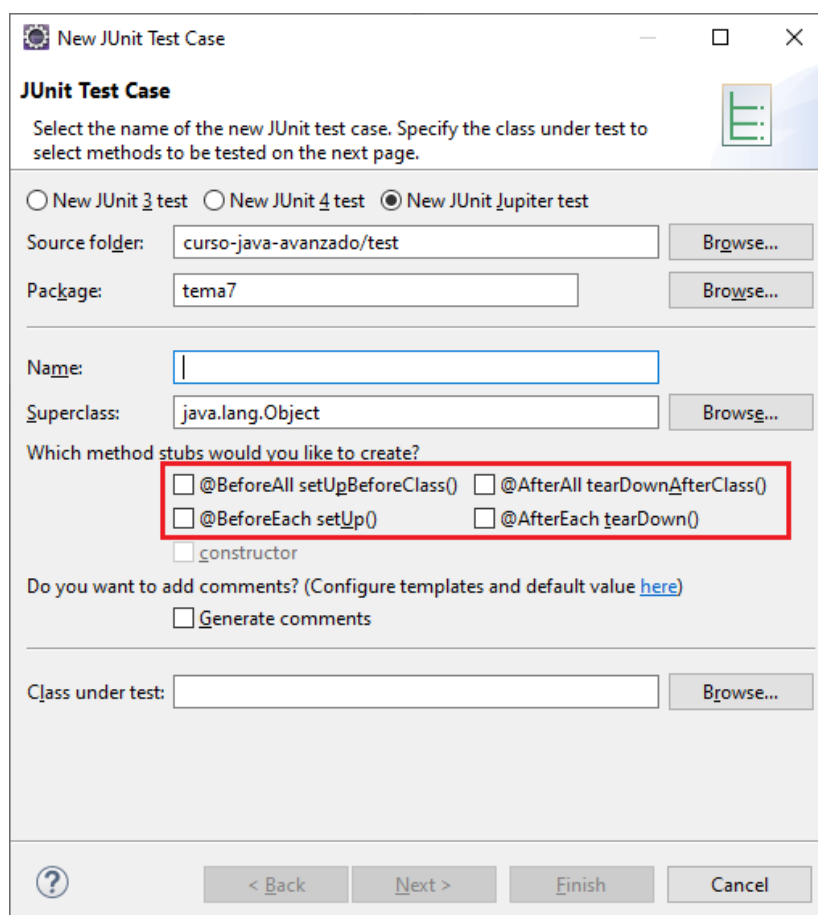
```
@Test
void testNullException() {
    String s = null;
    assertThrows(NullPointerException.class, () -> CuentaVocales.cuentaVocales(s));
}
```

El primer argumento de `assertThrows` es la clase de la excepción esperada, y el segundo argumento debe ser un objeto de tipo `Executable`, que es un interfaz funcional que podemos implementar con una expresión lambda sin argumentos.

Hay que tener en cuenta que podemos incluir varias sentencias `assert` consecutivas dentro del mismo método de prueba. Por ejemplo, supongamos un método que nos devuelve una colección: dentro del mismo test podríamos comprobar consecutivamente que la colección devuelta no sea null, luego que sea de tipo `ArrayList`, luego que tenga un tamaño determinado, etc. En cuanto una de las condiciones no se cumpla, el método finalizará con resultado `Failure`.

4. Métodos de ciclo de vida

Cuando creamos un Test Case desde Eclipse vemos que nos ofrece varias opciones:



Estos métodos cumplen las siguientes funciones:

- `setUpBeforeClass`: crea un método estático con la anotación `@BeforeAll`. Los métodos con esta anotación se ejecutan una única vez antes de que se empiecen a ejecutar los métodos de test. Por ejemplo podría ser abrir una conexión a base de datos, y luego todos los métodos de test utilizan la misma conexión (esta conexión debe declararse como `static` para que pueda ser utilizada por todos los tests, ya que cada ejecución de un test crea una nueva instancia de la clase).
- `tearDownAfterClass`: crea un método estático con la anotación `@AfterAll`. Los métodos con esta anotación se ejecutan una única vez después de que se hayan ejecutado todos los métodos de test. Por ejemplo podría ser cerrar la conexión de base de datos del ejemplo anterior.
- `setUp`: crea un método con la anotación `@BeforeEach`. Los métodos con esta anotación se ejecutan antes de cada uno de los métodos de test. Por ejemplo podría ser mostrar un mensaje por consola o inicializar ciertos atributos.
- `tearDown`: crea un método con la anotación `@AfterEach`. Los métodos con esta anotación se ejecutan después de cada uno de los métodos de test. Por ejemplo podría ser mostrar cierto texto por la consola.

A estos métodos se les llama métodos de ciclo de vida (*lifecycle methods*).

5. Las pruebas unitarias en las metodologías de desarrollo de software

Las metodologías de desarrollo de software modernas tienden a incorporar las pruebas unitarias a lo largo de todo el ciclo de vida. Por ejemplo, a veces se exige a los desarrolladores que todas las clases tengan su correspondiente prueba unitaria. Algunos IDEs permiten obtener “la tasa de cobertura” de las pruebas unitarias, es decir, qué porcentaje de clases y métodos tienen al menos una prueba unitaria.

Al diseñar las pruebas se suelen considerar dos enfoques:

- Pruebas de caja negra: las pruebas se diseñan teniendo en cuenta la definición del método, es decir, que el método cumple con lo que especifica en su definición de cara al exterior, sin tener en cuenta cómo está implementado. En este caso por ejemplo si un método define que lanza una excepción en determinado caso, se realizará una prueba para comprobarlo.
- Pruebas de caja blanca: las pruebas se diseñan teniendo en cuenta cómo está implementado el método, y el objetivo de estas pruebas debe ser tener una cobertura del código del 100%, es decir, que tras ejecutar todas las pruebas se haya ejecutado cada sentencia de código al menos una vez. Por ejemplo, si dentro de un método tengo una sentencia `if ... else if ... else if ...`, con tres situaciones posibles, deberé diseñar tres pruebas para que cada una de ellas ejecute una de las tres condiciones posibles.

Como vemos diseñar correctamente las pruebas unitarias es una tarea que requiere de bastante tiempo y dedicación, no obstante todo el tiempo invertido se recupera ya que ahorramos tiempo en incidencias que no se van a producir, y ahorraremos tiempo en el mantenimiento evolutivo.

Las pruebas unitarias se incorporan incluso en los entornos de integración continua. Por ejemplo se puede forzar a que antes de que un desarrollador pueda subir cambios al repositorio de código fuente, el código no sólo compile sino que también pase todas las pruebas unitarias, y en caso de que no sea así rechazar los cambios en el repositorio.

No obstante, debe tenerse en cuenta que las pruebas unitarias no garantizan un software libre de errores, ya que no evitan un mal diseño. Tampoco suelen utilizarse para pruebas de alto nivel, sino más bien para probar que cada módulo cumple su función. Así pues deben considerarse como un elemento más a tener en cuenta para conseguir un software robusto y de calidad, pero no el único.

6. Ejercicio Resuelto

Se nos pide desarrollar un método que calcule la edad en un momento dado, a partir de la fecha de nacimiento. El método tendrá como entrada la fecha de nacimiento y la fecha para la que se quiere calcular la edad (ambos de tipo `java.util.Date`), y como salida la edad en años.

Desarrollamos, dentro de una clase `Utilidades`, el siguiente método:

```
/**
 * Calcula la edad en una determinada fecha.
 *
 * @param fechaNacimiento la fecha de nacimiento
 * @param fecha la fecha en la que se quiere calcular la edad
 * @return la edad
 */
public static int calculaEdad(Date fechaNacimiento, Date fecha) {
    return (int) ((fecha.getTime() - fechaNacimiento.getTime()) /
        (365.25*24*60*60*1000));
}
```

Este código calcula el número de milisegundos transcurridos y divide por el número de milisegundos de un año. Para tener en cuenta el efecto de los bisiestos considera que un año tiene 365.25 días. ¿Funcionará correctamente este código?

El alumno deberá desarrollar pruebas unitarias de caja negra para cada uno de los siguientes casos y averiguar si alguno falla.

Todos los test tendrán la misma fecha de nacimiento (08/08/1978) y se comprobará si calcula la edad correctamente para las siguientes fechas:

- 01/01/2022 -> la edad debe ser 43
- 07/08/2022 (día anterior -> 43)
- 08/08/2022 (mismo día -> 44)
- 09/08/2022 (día posterior -> 44)
- 31/12/2022 -> 44
- 07/08/1979 -> 0
- 08/08/1979 -> 1
- 09/08/1979 -> 1

Por ejemplo para la primera prueba el test sería algo así:

```
class FechaNacimientoTest {

    SimpleDateFormat dateFormat = new SimpleDateFormat("dd/MM/yyyy");

    @Test
    void testAnterior() throws ParseException {
        assertEquals(
            43,
            Utilidades.calculaEdad(
                dateFormat.parse("08/08/1978"),
                dateFormat.parse("01/01/2022")));
    }
}
```

}

De las ocho pruebas indicadas una de ellas fallará, mostrando que el algoritmo no es correcto. El alumno deberá implementar todos los casos de prueba para averiguar cuál es.

NOTA: este ejercicio no es evaluable y no debe ser entregado por el alumno. El alumno sabrá que su solución es correcta cuando haya detectado que uno de los ocho casos indicados no calcula la edad correctamente.

Téngase en cuenta que no se trata de editar el caso de prueba hasta dar con el caso que falla, sino de crear ocho métodos de prueba y ejecutar los ocho a la vez. De esta manera una vez implementadas todas las pruebas unitarias sería muy fácil cambiar el algoritmo de cálculo de la edad y comprobar si el nuevo algoritmo es o no correcto, ejecutando de nuevo las pruebas. En este caso, aún sería conveniente añadir más casos de prueba, por ejemplo para alguien nacido un 29 de febrero ver la edad a 1 de marzo de un año no bisiesto, etc. Siempre hay que tratar de ser lo más exhaustivo posible.